The Journal of Open Source Software

LightKrylov: Lightweight implementation of Krylov subspace techniques in modern Fortran

J. Simon Kern [1*], **Ricardo S. Frantz** [1*], and **Jean-Christophe Loiseau** [1*¶]

1 Arts et Métiers Institute of Technology ¶ Corresponding author * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 20 October 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

LightKrylov is a Fortran package providing a collection of Krylov methods for large-scale linear algebra. Direct methods tend to have a computational cost scaling as $\mathcal{O}(n^3)$ making them impractical for large-scale problems. Even when A is sparse, computing matrix factorizations can incur an $\mathcal{O}(n^2)$ storage cost in the worst case by introducing fill-in. In contrast, Krylov methods only need a function computing the matrix-vector product $u \leftarrow Av$ (or $u \leftarrow A^H v$) to iteratively construct a *Krylov subspace* (Krylov, 1931) from which the solutions can be extracted, see Ipsen & Meyer (1998), Saad (2003) and to Frantz et al. (2023).

Statement of need

A collection of Krylov-based algorithms in pure modern Fortran

LightKrylov provides Fortran users with SciPy-inspired interfaces to widely used Krylov techniques, including:

- **Linear systems** - Conjugate Gradient (CG), Generalized Minimal Residual method (GMRES) and Flexible GMRES.
- **Spectral decomposition** - Arnoldi method (with Krylov-Schur restart) for non-Hermitian operators, Lanczos tridiagonalization for Hermitian ones.
- **SVD** - Golub-Kahan bidiagonalization.

Libraries exposing more methods exist in other languages, e.g., Krylov.jl (Montoison & Orban, 2023) or PETSc (Balay et al., 2015). Integrating these into existing Fortran codes can however present challenges (e.g. dependency management). In contrast, LightKrylov is a pure Fortran alternative, compliant with the 2018 standard, and requiring only the Fortran standard library stdlib as an dependency. Finally, its build process relies on the Fortran package manager fpm, facilitating its integration with the modern Fortran ecosystem.

A focus on abstract linear operators and abstract vectors

Krylov methods can be implemented without explicit reference to the data structure used to represent a vector or linear operator, nor to how the matrix-vector product is implemented. To do so, LightKrylov uses modern Fortran abstract types. A stripped-down version of the abstract vector type is shown below.

```fortran
type, abstract :: abstract_vector_rdp
  contains
    ! Abstract procedure to compute the scalar-vector product.
    procedure(abstract_scal_rdp), pass(self), deferred :: scal
    ! Abstract procedure to compute y = alpha*x + beta*y.
    procedure(abstract_axpby_rdp), pass(self), deferred :: axpby
```

```
    ! Abstract procedure to compute the vector dot product.
    procedure(abstract_dot_rdp), pass(self), deferred :: dot
end type
```

The type-bound procedures cover the basic operations on vectors: scalar-vector product, linear combination of two vectors, and the dot product. These operations are the essential building blocks required by Krylov algorithms. Their signatures follow, to the extent possible, the BLAS standard. For instance, the `abstract_axpby_rdp` interface reads

```
abstract interface
  subroutine abstract_axpby_rdp(alpha, vec, beta, self)
    double precision, intent(in)    :: alpha, beta
    class(abstract_vector_rdp), intent(in)    :: vec
    class(abstract_vector_rdp), intent(inout) :: self
  end subroutine
end interface
```

mimicking the signature of the (extended) BLAS-1 subroutine axpby. Abstract linear operators are defined similarly, with two type-bound procedures required to implement the matrix-vector and transpose matrix-vector product. Using `abstract` types enables us to focus on the high-level implementation of the different algorithms while leaving the performance-critical details to the users. In addition, `LightKrylov` exposes abstract types for preconditioners, as well as a Newton-GMRES solver for nonlinear systems. After extending these `abstract` types for their application, one can solve linear systems or compute eigenvalues as easily as `call gmres(A, b, x, info)` or `call eigs(A, V, lambda, residuals, info)`.

Performance in a production-ready open-source codebase

`LightKrylov` has been integrated into neklab, a toolbox for stability and bifurcation analysis for the spectral element solver `Nek5000`. The abstract vector interface allows direct use of `Nek5000`'s distributed data structures, and the pure-Fortran nature facilitated integration with its existing build system, demonstrating the library's suitability for large-scale HPC applications.

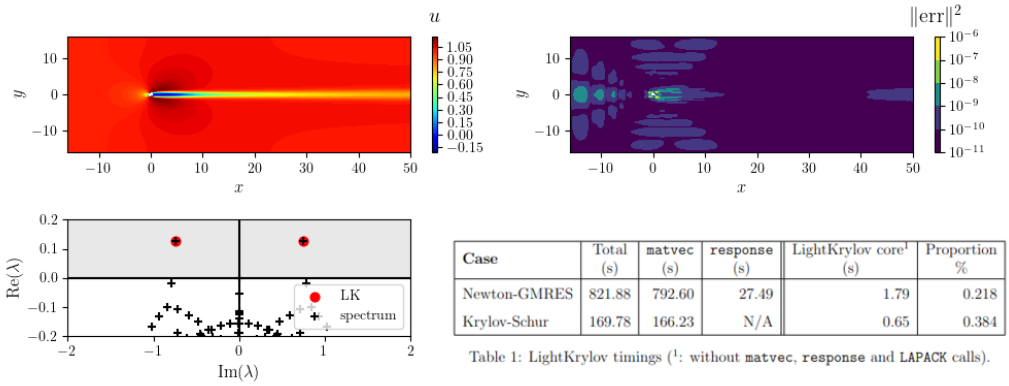
Hydrodynamic stability of an unstable fixed point of the nonlinear Navier-Stokes equations

Using the two-dimensional flow past a circular cylinder at Reynolds number of 100, we showcase the efficient integration of `LightKrylov` and `Nek5000` and validate the results with algorithms provided by the `KTH Framework` toolbox (Massaro et al., 2024) based on the same solver. Discretization of the governing equations leads to systems with approximately 175,000 degrees of freedom. All computations were run in parallel on 12 Intel Core Ultra 7 processors and the numerical settings are identical for both libraries.

The unstable fixed point of the nonlinear Navier-Stokes equations is computed using both `LightKrylov`'s *time-stepper*-based Newton-GMRES solver and the selected frequency damping implementation from `KTH Framework` . Likewise, the leading eigenpair of the corresponding linearized Navier-Stokes operator is computed using `LightKrylov`'s implementation of the Krylov-Schur algorithm and the `KTH Framework`'s wrapper for `ARPACK`.

A visual comparison is provided in Figure 1 showing excellent agreement. The table in the lower-right panel of Figure 1 summarizes the wall-clock times of the `neklab` computations. Isolating the intrinsic cost of the algorithms in `LightKrylov` from the cost of the calls to LAPACK and the linear and nonlinear Navier-Stokes solvers (`matvec` and `response`, respectively) shows that extended `abstract` types and object-oriented programming in Fortran incurs a negligible computational overhead for such large-scale applications.

DRAFT



Case	Total (s)	matvec (s)	response (s)	LightKrylov core[1] (s)	Proportion %
Newton-GMRES	821.88	792.60	27.49	1.79	0.218
Krylov-Schur	169.78	166.23	N/A	0.65	0.384

Table 1: LightKrylov timings ([1]: without matvec, response and LAPACK calls).

Figure 1: Validation of `LightKrylov`: Newton-GMRES and `eigs`. The top row depicts the streamwise velocity of the unstable solution computed using `LightKrylov` and the pointwise difference with the reference one computed with `KTH Framework`.

Perspectives

Despite being in its early development stage, `LightKrylov` has already been used in production runs on dozens of processors. It has also been interfaced with neko, a modernized implementation of `Nek5000` running on GPUs, and is currently being interfaced with `dNami`, a high-performance source code generator for hyperbolic partial differential equations.

Ongoing development efforts include:

- **Build system:** `LightKrylov` relies on the Fortran package manager `fpm` for its build. While it ensures nice integration into the modern Fortran ecosystem, it also limits its usage to codes built using `fpm`. A build system based on `CMake` for easier integration into non-`fpm` codes is in development.
- **Krylov processes:** `LightKrylov` currently provides implementations for three of the most important Krylov processes. Future releases will include the non-Hermitian Lanczos and two-sided Arnoldi factorizations, as well as the Saunders-Simon-Yip process (Saunders et al., 1988).
- **Iterative solvers:** The integration of new Krylov processes will enable us to provide an extended list of solvers, including `MINRES`, `CGNE`, `LSQR`, and `LSMR`.

Acknowledgements

We acknowledge the financial support of the French National Agency for Research (ANR) through the ANR-33-CE46-0008-CONMAN grant agreement. We also thank the fortran-lang community for the development of `stdlib`, and in particular Frederico Perini, Jeremie Vandenplas, and Jose Alvez for their work on the `stdlib_linalg` module.

References

Balay, S., Abhyankar, S., Adams, M. F., Brown, J., Brune, P., Buschelman, K., Dalcin, L., Eijkhout, V., Gropp, W. D., Kaushik, D., Knepley, M. G., McInnes, L. C., Rupp, K., Smith, B. F., Zampini, S., & Zhang, H. (2015). *PETSc web page*. http://www.mcs.anl.gov/petsc

Frantz, R. A. S., Loiseau, J.-Ch., & Robinet, J.-Ch. (2023). Krylov Methods for Large-Scale Dynamical Systems: Application in Fluid Dynamics. *Applied Mechanics Reviews*, *75*(3). https://doi.org/10.1115/1.4056808

DRAFT

Ipsen, I. C. F., & Meyer, C. D. (1998). The Idea Behind Krylov Methods. *The American Mathematical Monthly*, *105*(10), 889–899. https://doi.org/10.1080/00029890.1998.12004985

Krylov, A. N. (1931). On the numerical solution of the equation by which, in technical matters, frequencies of small oscillations of material systems are determined. *Izvestija AN SSSR (News of Academy of Sciences of the USSR), Otdel. Mat. I Estest. Nauk*, *7*(4), 491–539.

Massaro, D., Peplinski, A., Stanly, R., Mirzareza, S., Lupi, V., Mukha, T., & Schlatter, P. (2024). A comprehensive framework to enhance numerical simulations in the spectral-element code Nek5000. *Computer Physics Communications*, *302*, 109249. https://doi.org/10.1016/j.cpc.2024.109249

Montoison, A., & Orban, D. (2023). Krylov.jl: A julia basket of hand-picked krylov methods. *Journal of Open Source Software*, *8*(89), 5187. https://doi.org/10.21105/joss.05187

Saad, Y. (2003). *Iterative Methods for Sparse Linear Systems*. Society for Industrial and Applied Mathematics. https://doi.org/10.1137/1.9780898718003

Saunders, M. A., Simon, H. D., & Yip, E. L. (1988). Two conjugate-gradient-type methods for unsymmetric linear equations. *SIAM Journal on Numerical Analysis*, *25*(4), 927–940. https://doi.org/10.1137/0725052

DRAFT